

รายงานโครงงาน
แอปพลิเคชันสั่งอาหารสำหรับลูกค้าที่นั่งในร้าน

จัดทำโดย

อิสระ กุมพล 6721651947

วรเมธ เล็กทรัพย์ 6721651785

ขวัญนนท์ จิตติเจริญรักษ์ 6721651467

เสนอ

อ.ดร.วรัทภพ ธวัชรสุวรรณ

รายงานนี้เป็นส่วนหนึ่งของรายวิชา

01418342-69

การออกแบบและพัฒนาโปรแกรมประยุกต์สำหรับอุปกรณ์เคลื่อนที่

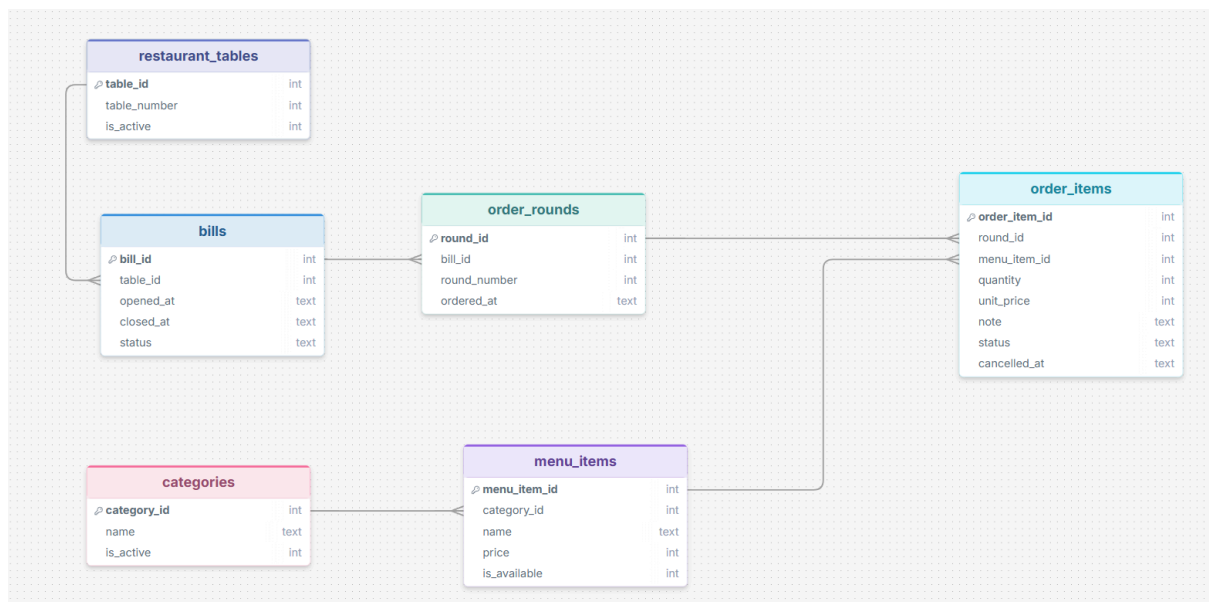
Mobile Application Design and Development

1.แผนผังฐานข้อมูล ฐานข้อมูลของระบบประกอบด้วย 6 ตาราง ได้แก่

categories,
menu_items,
restaurant_tables,
bills,
order_rounds,
order_items

โดยแต่ละตารางกำหนด Primary Key เพื่อใช้ระบุข้อมูลแต่ละรายการ และใช้ Foreign Key ในการเชื่อมโยงข้อมูลที่เกี่ยวข้องกัน

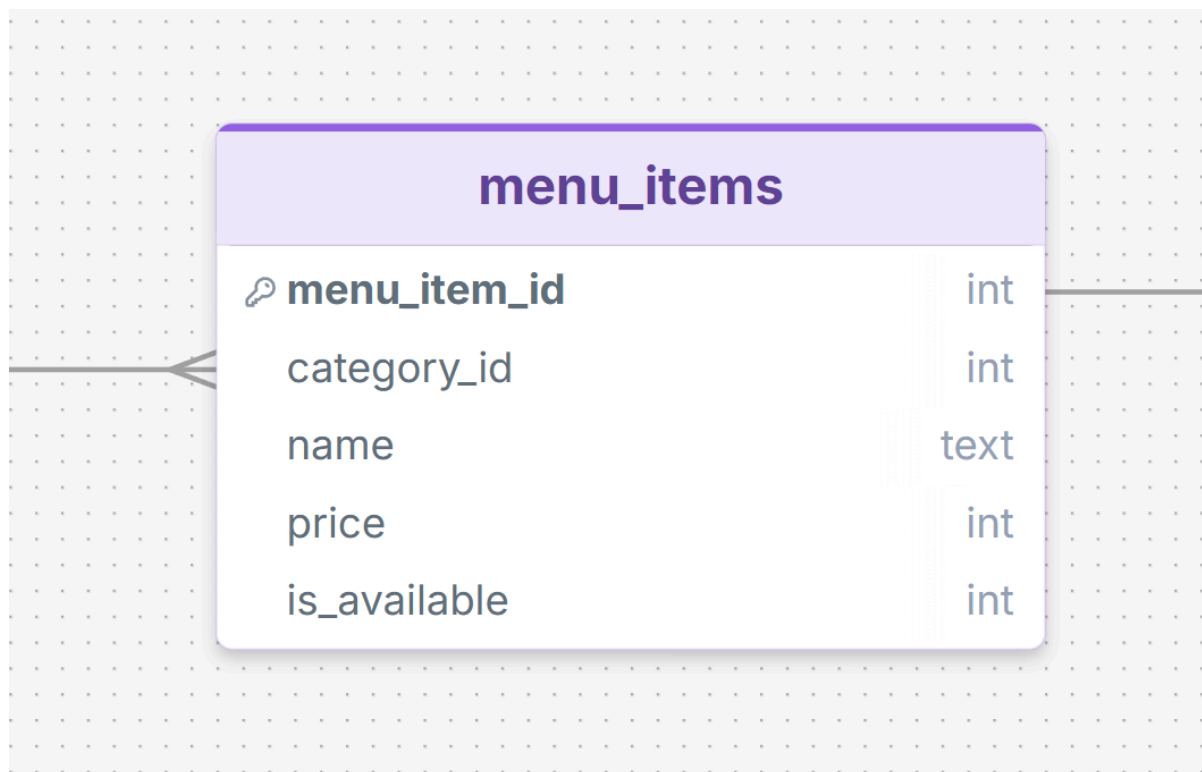
ความสัมพันธ์ของข้อมูลเริ่มจากโต๊ะในร้าน (restaurant_tables) เชื่อมกับบิล (bills) แต่ละบิลสามารถมีรอบการสั่งหลายรอบ (order_rounds) และในแต่ละรอบสามารถมีรายการอาหารหลายรายการ (order_items) ส่วนข้อมูลเมนู (menu_items) จะเชื่อมกับหมวดหมู่อาหาร (categories) และเชื่อมกับรายการที่ลูกค้าสั่งผ่าน menu_item_id ทำให้ระบบสามารถตรวจสอบได้ว่ารายการอาหารแต่ละรายการเป็นเมนูใด อยู่ในรอบการสั่งใด อยู่ในบิลใด และมาจากโต๊ะใด



2.เหตุผลของการแยกตาราง

ฐานข้อมูลของระบบถูกแบ่งออกเป็น 6 ตาราง ได้แก่ categories, menu_items, restaurant_tables, bills, order_rounds และ order_items เนื่องจากข้อมูลแต่ละกลุ่มมีหน้าที่และลักษณะการใช้งานแตกต่างกัน หากนำข้อมูลทั้งหมดมารวมไว้ในตารางเดียว จะทำให้เกิดข้อมูลซ้ำจำนวนมาก แก้ไขข้อมูลได้ยาก และอาจทำให้ข้อมูลไม่สอดคล้องกันได้ เช่น ชื่อหมวดหมู่ ชื่อเมนู หรือข้อมูลโต๊ะอาจต้องถูกบันทึกซ้ำทุกครั้งที่มีการสั่งอาหาร นอกจากนี้ระบบยังต้องรองรับการสั่งหลายรอบภายในบิลเดียวกัน จึงจำเป็นต้องแยกข้อมูลตามหน้าที่ของแต่ละส่วนอย่างชัดเจน

ตาราง menu_items



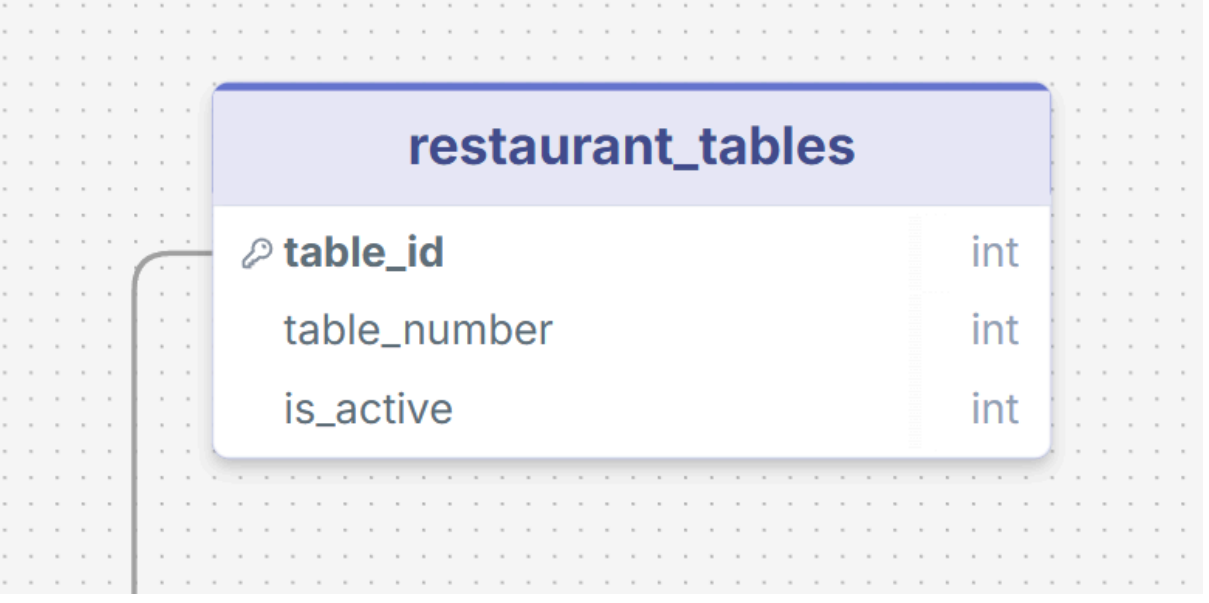
ตาราง menu_items ใช้เก็บข้อมูลประจำของเมนูอาหาร เช่น ชื่ออาหาร ราคา หมวดหมู่ และสถานะว่ายังสามารถขายได้หรือไม่ โดยข้อมูลเหล่านี้เป็นข้อมูลของ “เมนู” ไม่ใช่ข้อมูลของการสั่งอาหารแต่ละครั้ง จึงไม่ควรรวมอยู่กับ order_items

ตัวอย่างเช่น เมนู “ข้าวกะเพรา” อาจถูกลูกค้าหลายโต๊ะสั่ง และสามารถถูกสั่งหลายครั้งในหลายบิล หากเก็บชื่ออาหารและข้อมูลเมนูทั้งหมดเข้าไปใน order_items ทุกครั้งที่มีการสั่ง จะทำให้ข้อมูลซ้ำจำนวนมาก

การแยก menu_items ออกมาเป็นตารางหลักของเมนู ทำให้รายการที่ถูกสั่งสามารถอ้างอิงเมนูผ่าน menu_item_id ได้ โดยไม่ต้องเก็บข้อมูลเมนูทั้งหมดเข้าไปในทุกออเดอร์ และทำให้สามารถแก้ไขข้อมูลเมนู เช่น ชื่อหรือสถานะการขาย ได้จากจุดเดียว

อย่างไรก็ตาม ราคาที่ใช้จริงตอนลูกค้าสั่งจะถูกเก็บไว้ใน order_items.unit_price อีกครั้ง เพื่อรักษาราคาของรายการที่สั่งไว้แล้ว ซึ่งจะอธิบายรายละเอียดในหัวข้อการเก็บราคา

ตาราง restaurant_tables



The diagram shows a database table named 'restaurant_tables' with a light purple header. Below the header, the table's columns are listed in a white box with a light purple border. The columns are 'table_id', 'table_number', and 'is_active'. 'table_id' is marked as the primary key with a key icon. The data types for each column are listed to the right: 'int' for 'table_id', 'int' for 'table_number', and 'int' for 'is_active'. A line connects the table name to the table structure box.

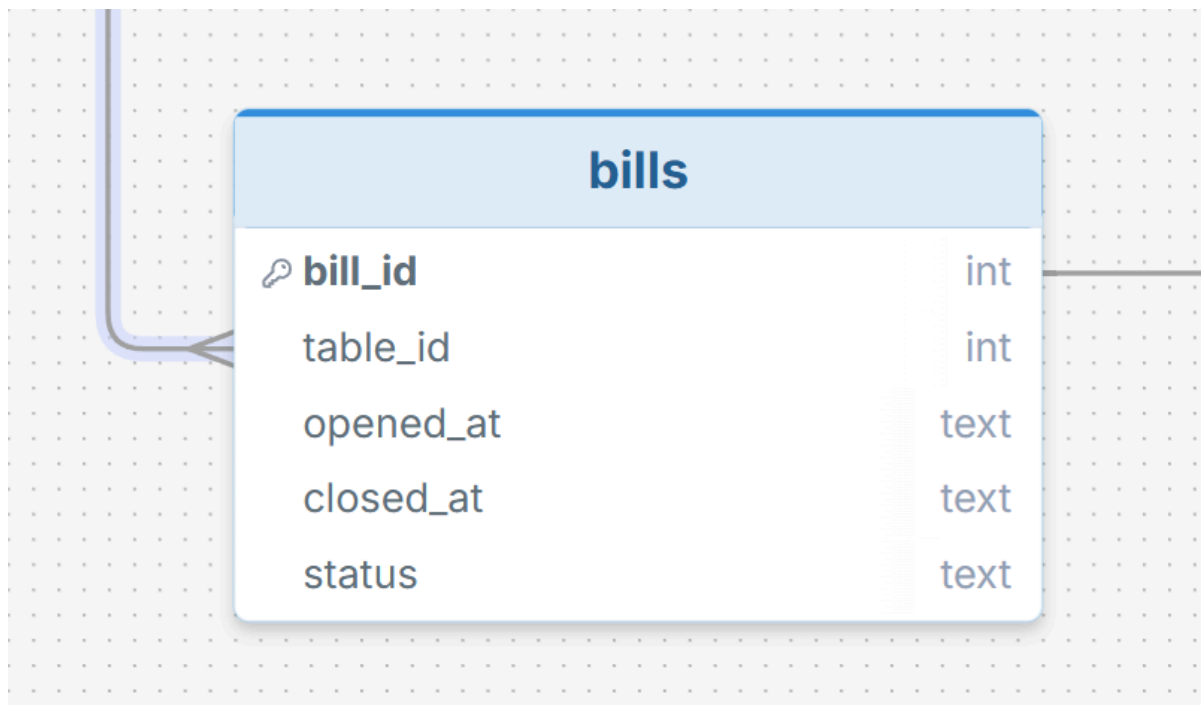
restaurant_tables	
🔑 table_id	int
table_number	int
is_active	int

ตาราง restaurant_tables ใช้เก็บข้อมูลโต๊ะภายในร้าน เช่น table_number และสถานะว่าโต๊ะนั้นยังเปิดใช้งานอยู่หรือไม่ ข้อมูลโต๊ะเป็นข้อมูลค่อนข้างคงที่ และสามารถถูกใช้งานซ้ำได้กับบิลหลายบิลในช่วงเวลาต่างกัน จึงไม่ควรนำข้อมูลโต๊ะไปรวมกับตาราง bills

ตัวอย่างเช่น โต๊ะหมายเลข 5 ในวันนี้อาจมีลูกค้ากลุ่มหนึ่งเปิดบิลและปิดบิลไปแล้ว หลังจากนั้นลูกค้ากลุ่มใหม่สามารถมาใช้โต๊ะหมายเลข 5 และเปิดบิลใหม่ได้อีก ดังนั้นโต๊ะหนึ่งโต๊ะจึงสามารถมีบิลหลายบิลในช่วงเวลาที่ต่างกัน

ระบบจึงเก็บข้อมูลโต๊ะไว้ใน restaurant_tables เพียงครั้งเดียว และให้ bills.table_id เชื่อมกลับมาที่โต๊ะนั้น วิธีนี้ช่วยให้สามารถเก็บประวัติของบิลแต่ละรอบการใช้งานได้ โดยไม่ต้องสร้างข้อมูลโต๊ะใหม่ทุกครั้งที่มีลูกค้าเข้ามาใช้โต๊ะเดิม

ตาราง bills



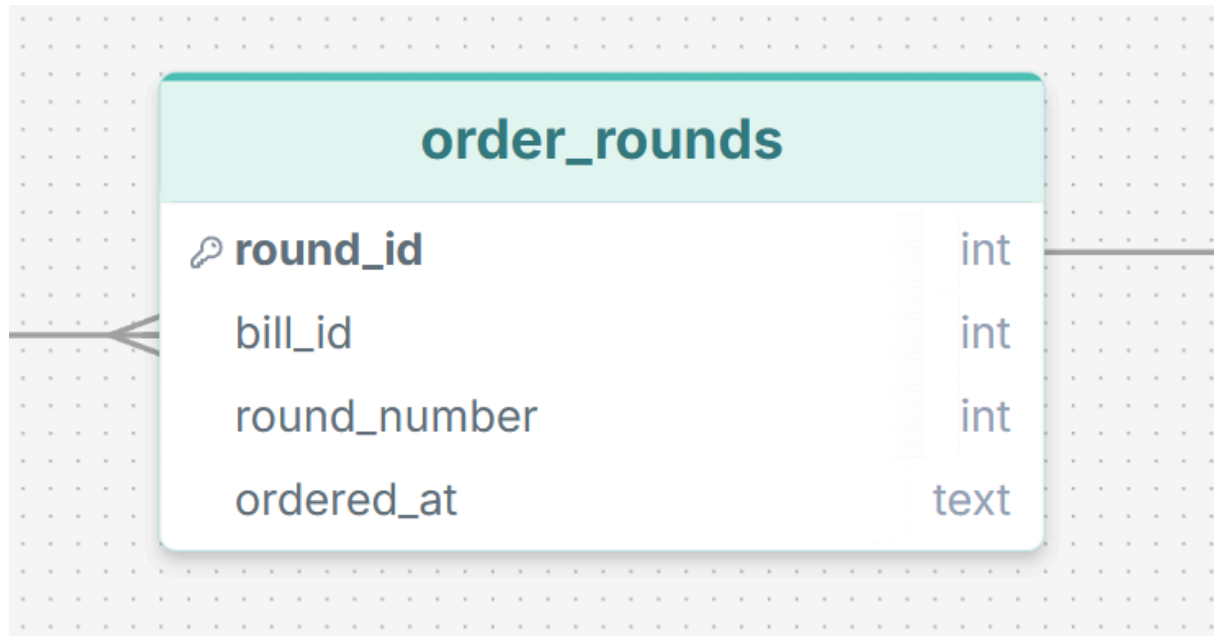
ตาราง bills ใช้แทนการเข้าใช้โต๊ะหนึ่งครั้งตั้งแต่เริ่มเปิดบิลจนถึงปิดบิล โดยเก็บว่าเป็นบิลของโต๊ะใด เปิดเมื่อใด ปิดเมื่อใด และมีสถานะเป็น open หรือ closed

เหตุผลที่ต้องแยก bills ออกจาก restaurant_tables เพราะ “โต๊ะ” และ “บิล” เป็นข้อมูลคนละประเภท โต๊ะเป็นทรัพยากรของร้านที่มีอยู่ตลอด แต่บิลเป็นเหตุการณ์ที่เกิดขึ้นแต่ละครั้งเมื่อมีลูกค้ามาใช้โต๊ะ

ถ้าเก็บข้อมูลบิลไว้ในตารางโต๊ะโดยตรง เช่น เก็บเวลาเปิด เวลาออก และสถานะบิลไว้ใน restaurant_tables เมื่อมีลูกค้ากลุ่มใหม่มาใช้โต๊ะเดิม ข้อมูลบิลเก่าจะต้องถูกเขียนทับ ทำให้ไม่สามารถเก็บประวัติการใช้งานหรือเรียกดูบิลย้อนหลังได้

การแยก bills ออกมา ทำให้โต๊ะหนึ่งโต๊ะสามารถมีบิลได้หลายบิลในช่วงเวลาต่างกัน และเมื่อปิดบิลแล้ว ข้อมูลของบิลเก่ายังคงอยู่ ขณะที่โต๊ะสามารถเปิดบิลใหม่ได้ตาม Requirement ของระบบ

ตาราง order_rounds



ตาราง `order_rounds` เป็นส่วนสำคัญของระบบนี้ เพราะโจทย์กำหนดให้ลูกค้าสามารถสั่งอาหารเพิ่มได้หลายรอบภายในบิลเดียวกัน ดังนั้นจึงไม่สามารถเชื่อมรายการอาหารที่สั่งเข้ากับ `bills` โดยตรงเพียงอย่างเดียวได้

ตัวอย่างเช่น โต๊ะ 3 เปิดบิลหนึ่งบิล แล้วสั่งอาหารครั้งแรก 3 รายการ หลังจากรับประทานไปแล้วอาจสั่งเครื่องดื่มเพิ่มเป็นรอบที่ 2 และสั่งของหวานเพิ่มเป็นรอบที่ 3 ทั้งหมดนี้ยังเป็นบิลเดียวกัน แต่เป็นคนละรอบการสั่ง

ตาราง `order_rounds` จึงทำหน้าที่เป็นตัวกลางระหว่าง `bills` และ `order_items` โดยเก็บ `bill_id`, `round_number` และ `ordered_at` ทำให้ระบบสามารถทราบได้ว่าการสั่งแต่ละรอบเป็นของบิลใด เป็นรอบที่เท่าใด และถูกส่งเข้าครัวเมื่อเวลาใด

หากไม่มีตาราง `order_rounds` แล้วเก็บ `bill_id` ไว้ใน `order_items` โดยตรง ระบบจะรู้เพียงว่าอาหารแต่ละรายการอยู่ในบิลใด แต่จะไม่สามารถแยกได้อย่างชัดเจนว่ารายการไหนถูกสั่งพร้อมกันในรอบที่ 1, 2 หรือ 3 ซึ่งจะกระทบทั้งหน้าสรุปบิลและหน้าครัว

การแยกตารางนี้ยังช่วยลดข้อมูลซ้ำ เพราะข้อมูล เช่น `round_number` และ `ordered_at` ถูกเก็บเพียงครั้งเดียวต่อหนึ่งรอบ ไม่ต้องบันทึกซ้ำในทุก `order_items` ที่อยู่ในรอบเดียวกัน

ตาราง order_items

order_items	
 order_item_id	int
round_id	int
menu_item_id	int
quantity	int
unit_price	int
note	text
status	text
cancelled_at	text

ตาราง order_items ใช้เก็บรายละเอียดจริงของอาหารแต่ละรายการที่ลูกค้าสั่ง เช่น เมนูที่สั่ง จำนวน ราคา ณ เวลาที่สั่ง หมายเหตุ และสถานะการทำอาหาร

ตารางนี้ต้องแยกออกจาก order_rounds เพราะหนึ่งรอบการสั่งสามารถมีอาหารได้หลายรายการ เช่น รอบที่ 1 อาจมีข้าวกะเพรา 2 จาน ต้มยำ 1 หม้อ และน้ำเปล่า 3 ขวด หากนำข้อมูลรายการอาหารไปรวมไว้ใน order_rounds จะไม่สามารถเก็บอาหารหลายรายการในหนึ่งรอบได้อย่างเหมาะสม หรืออาจต้องสร้างคอลัมน์ลักษณะ item1, item2, item3 ซึ่งจะไม่ยืดหยุ่น และไม่สามารถทราบล่วงหน้าได้ว่าลูกค้าจะสั่งกี่รายการ

ดังนั้น order_rounds จึงเก็บข้อมูลระดับ “หนึ่งรอบการสั่ง” ส่วน order_items เก็บข้อมูลระดับ “แต่ละรายการอาหารในรอบนั้น” โดยใช้ round_id เป็น Foreign Key เชื่อมกลับไปยังรอบการสั่ง

นอกจากนี้ order_items ยังมี status แยกในแต่ละรายการ เพราะอาหารในรอบเดียวกันอาจมีสถานะไม่เหมือนกัน เช่น ข้าวกะเพราอาจอยู่สถานะ served แล้ว แต่ต้มยำยังอยู่สถานะ cooking การเก็บสถานะไว้ที่ระดับรายการจึงทำให้ครัวสามารถเปลี่ยนสถานะอาหารแต่ละรายการได้อย่างอิสระ

3. การเก็บราคา

ระบบใช้วิธีบันทึกราคาต่อหน่วย ณ เวลาที่สั่งอาหารไว้ในฟิลด์ unit_price ของตาราง order_items โดยแยกจากราคาปัจจุบันของเมนูในตาราง menu_items เมื่อร้านเปลี่ยนราคาเมนู

ราคาที่บันทึกไว้ในรายการสั่งซื้อเดิมจึงไม่เปลี่ยนแปลงตาม การแสดงราคาและค่านวนยอดรวมของบิลจะใช้ unit_price คูณกับจำนวนที่สั่ง และไม่นำรายการที่ยกเลิกมารวมยอด

ตัวอย่างเช่น เมื่อวานลูกค้าสั่งผัดไทยราคา 50 บาท ระบบจะบันทึกราคา 50 บาทไว้ในรายการสั่งซื้อ แม้วันนี้ร้านปรับราคาเป็น 60 บาท บิลของเมื่อวานก็ยังคำนวณจากราคาเดิมที่ 50 บาท

เหตุผลที่เลือกวิธีนี้คือ เพื่อให้บิลคือราคาที่ลูกค้าสั่งจริง ป้องกันยอดเงินย้อนหลังเปลี่ยนแปลงเมื่อแก้ไขราคาเมนู และช่วยให้ตรวจสอบประวัติการขายได้อย่างถูกต้อง

4. การรองรับหลายรอบการสั่ง

ระบบแยกข้อมูลออกเป็นสามตาราง ได้แก่ bills สำหรับเก็บบิล order_rounds สำหรับเก็บรอบการสั่ง และ order_items สำหรับเก็บรายการอาหาร แต่ละรอบเชื่อมกับบิลผ่าน bill_id และแต่ละรายการอาหารเชื่อมกับรอบผ่าน round_id จึงสามารถระบุได้ว่ารายการอาหารแต่ละรายการอยู่ในรอบใดและบิลใด

เมื่อลูกค้าสั่งอาหารเพิ่มเติม ระบบจะใช้บิลเดิมที่ยังเปิดอยู่ แล้วสร้างรอบใหม่ โดยเพิ่ม round_number จากรอบล่าสุดทีละหนึ่ง พร้อมบันทึกเวลาสั่งใน ordered_at และกำหนดไม่ให้เลขรอบซ้ำกันภายในบิลเดียวกัน เช่น บิลหนึ่งมีรอบที่ 1 สั่งผัดไทย 2 จาน และรอบที่ 2 สั่งน้ำ 2 แก้ว ทั้งสองรอบจะแสดงแยกกัน แต่รวมยอดชำระในบิลเดียวกัน

การแยกรอบมีความจำเป็นเพื่อให้ตรวจสอบลำดับการสั่งและแยกรายการที่สั่งเพิ่มออกจากรายการเดิมได้ชัดเจน โดยเฉพาะกรณีสั่งเมนูเดิมซ้ำหลายครั้ง ช่วยลดความสับสนในการตรวจสอบรายการและติดตามสถานะอาหารของแต่ละรอบ

5. คำสั่ง SQL สามคำสั่งที่ซับซ้อนที่สุดในงาน

5.1 คำสั่งแสดงบิลทั้งหมด พร้อมค่านวนยอดสุทธิและจำนวนรายการ

ใช้ในไฟล์ AllBillsScreen.js เพื่อแสดงข้อมูลสรุปของแต่ละบิล

```

useFocusEffect(useCallback(() => {
  let active = true;
  db.getAllAsync(`
    SELECT
      b.bill_id,
      b.table_id,
      b.opened_at,
      b.closed_at,
      b.status,
      COALESCE(
        SUM(
          CASE
            WHEN i.status <> 'cancelled'
            THEN i.quantity * i.unit_price
            ELSE 0
          END
        ),
        0
      ) AS total,
      COUNT(i.order_item_id) AS item_count
    FROM bills b
    LEFT JOIN order_rounds r ON r.bill_id=b.bill_id
    LEFT JOIN order_items i ON i.round_id=r.round_id
    GROUP BY b.bill_id
    ORDER BY
      CASE b.status WHEN 'open' THEN 0 ELSE 1 END,
      b.opened_at DESC`
  )
})

```

```

SELECT
  b.bill_id,      -- เลือกรหัสบิล
  b.table_id,     -- เลือกรหัสโต๊ะของบิล
  b.opened_at,    -- เลือกเวลาเปิดบิล
  b.closed_at,    -- เลือกเวลาปิดบิล
  b.status,       -- เลือกสถานะบิล
  COALESCE(       -- หากผลรวมเป็น NULL ให้ใช้ค่า 0
    SUM(         -- รวมยอดเงินของรายการในบิล
      CASE
        WHEN i.status <> 'cancelled' -- ตรวจสอบว่ารายการไม่ได้ถูกยกเลิก
        THEN i.quantity * i.unit_price -- คำนวณจำนวนคูณราคาต่อชิ้น
        ELSE 0 -- รายการที่ยกเลิกคิดเป็น 0
      END -- สิ้นสุดเงื่อนไข CASE
    ),
    0 -- ค่าทดแทนเมื่อผลรวมเป็น NULL
  ) AS total,     -- ตั้งชื่อคอลัมน์ยอดสุทธิว่า total
  COUNT(i.order_item_id) AS item_count -- นับจำนวนแถวรายการอาหาร
FROM bills b      -- ใช้ตารางบิลเป็นตารางหลัก และตั้งชื่อย่อว่า b
LEFT JOIN order_rounds r ON r.bill_id = b.bill_id -- เชื่อมรอบการสั่งกับบิล

```

```

LEFT JOIN order_items i ON i.round_id = r.round_id -- เชื่อมรายการกับรอบ
GROUP BY b.bill_id -- จัดกลุ่มเพื่อคำนวณผลรวมแยกตามบิล
ORDER BY
  CASE b.status WHEN 'open' THEN 0 ELSE 1 END, -- เรียงบิลเปิดก่อนบิลปิด
  b.opened_at DESC; -- ภายในกลุ่มสถานะเดียวกัน เรียงบิลใหม่ก่อนบิลเก่า

```

5.2 คำสั่งแสดงคิวออเดอร์สำหรับครัว

ใช้ในไฟล์ QueOrderScreen.js เพื่อให้ครัวเห็นรายการอาหาร โต๊ะ รอบการสั่ง และสถานะของรายการ

```

useFocusEffect(useCallback(() => {
  let active = true;
  db.getAllAsync(`
    SELECT
      i.order_item_id,
      i.quantity,
      i.unit_price,
      i.note,
      i.status,
      r.round_number,
      b.table_id,
      m.name,
      r.ordered_at
    FROM order_items i
    JOIN order_rounds r ON r.round_id=i.round_id
    JOIN bills b ON b.bill_id=r.bill_id
    JOIN menu_items m ON m.menu_item_id=i.menu_item_id
    WHERE b.status='open'
    ORDER BY
      CASE i.status
        WHEN 'waiting' THEN 0
        WHEN 'cooking' THEN 1
        ELSE 2
      END,
      r.ordered_at,
      i.order_item_id`
  )

```

```

SELECT
  i.order_item_id, -- เลือกรหัสรายการอาหารที่สั่ง
  i.quantity,      -- เลือกจำนวนที่สั่ง
  i.unit_price,    -- เลือกราคาต่อหน่วยที่บันทึกตอนสั่ง
  i.note,          -- เลือกหมายเหตุของรายการ
  i.status,        -- เลือกสถานะของรายการอาหาร
  r.round_number,  -- เลือกเลขรอบการสั่ง
  b.table_id,      -- เลือกรหัสโต๊ะ

```

```

    m.name,          -- เลือกชื่อเมนูอาหาร
    r.ordered_at      -- เลือกเวลาที่สั่งรอบนั้น
FROM order_items i -- ใช้ตารางรายการอาหารที่สั่งเป็นตารางหลัก
JOIN order_rounds r ON r.round_id = i.round_id -- เชื่อมรายการกับรอบการสั่ง
JOIN bills b ON b.bill_id = r.bill_id -- เชื่อมรอบการสั่งกับบิล
JOIN menu_items m ON m.menu_item_id = i.menu_item_id -- เชื่อมเพื่อดึงชื่อเมนู
WHERE b.status = 'open' -- เลือกเฉพาะรายการที่อยู่ในบิลซึ่งยังเปิดอยู่
ORDER BY
    CASE i.status      -- กำหนดลำดับตามสถานะของรายการ
        WHEN 'waiting' THEN 0 -- รายการรอทำแสดงเป็นกลุ่มแรก
        WHEN 'cooking' THEN 1 -- รายการกำลังทำแสดงเป็นกลุ่มถัดไป
        ELSE 2           -- สถานะอื่นแสดงเป็นกลุ่มท้าย
    END,
    r.ordered_at,      -- ในกลุ่มเดียวกัน เรียงเวลาสั่งเก่าก่อนใหม่
    i.order_item_id;   -- หากเวลาตรงกัน เรียงรหัสรายการจากน้อยไปมาก

```

5.3 คำสั่งแสดงรายการและสถานะอาหารของโต๊ะที่เลือก

ใช้ในไฟล์ StatusOrderScreen.js เพื่อแสดงรายการอาหารของโต๊ะ โดยนำรอบการสั่งล่าสุดขึ้นก่อน

```

useFocusEffect(useCallback(() => {
  let active = true;
  (async () => {
    try {
      const rows = await db.getAllAsync(`
        SELECT
          i.order_item_id AS id,
          i.quantity,
          i.unit_price,
          i.note,
          i.status,
          r.round_number,
          m.name
        FROM order_items i
        JOIN order_rounds r ON r.round_id=i.round_id
        JOIN bills b ON b.bill_id=r.bill_id
        JOIN menu_items m ON m.menu_item_id=i.menu_item_id
        WHERE b.table_id=?
          AND b.status='open'
        ORDER BY
          r.round_number DESC,
          i.order_item_id`,

```

SELECT

```

i.order_item_id AS id, -- เลือกให้รายการ และตั้งชื่อคอลัมน์ผลลัพธ์ว่า id
i.quantity,           -- เลือกจำนวนที่สั่ง
i.unit_price,          -- เลือกราคาต่อหน่วยที่บันทึกตอนสั่ง
i.note,                -- เลือกหมายเหตุของรายการ
i.status,              -- เลือกสถานะของรายการอาหาร
r.round_number,        -- เลือกเลขรอบการสั่ง
m.name                 -- เลือกชื่อเมนูอาหาร
FROM order_items i      -- ใช้ตารางรายการอาหารที่สั่งเป็นตารางหลัก
JOIN order_rounds r ON r.round_id = i.round_id -- เชื่อมรายการกับรอบการสั่ง
JOIN bills b ON b.bill_id = r.bill_id -- เชื่อมรอบการสั่งกับบิล
JOIN menu_items m ON m.menu_item_id = i.menu_item_id -- เชื่อมเพื่อดึงชื่อเมนู
WHERE b.table_id = ?    -- เลือกเฉพาะโต๊ะตามรหัสที่โปรแกรมส่งเข้ามา
AND b.status = 'open' -- เลือกเฉพาะบิลที่ยังเปิดอยู่ของโต๊ะนั้น
ORDER BY
  r.round_number DESC, -- เรียงรอบการสั่งจากมากไปน้อย ให้รอบล่าสุดอยู่ก่อน
  i.order_item_id;    -- ภายในรอบเดียวกัน เรียงรหัสรายการจากน้อยไปมาก

```

6. ดัชนี

6.1 ดัชนีที่คอลัมน์ round_id ของตาราง order_items

สร้างดัชนีชื่อ idx_order_items_round_id ที่คอลัมน์ round_id ของตาราง order_items เพราะเวลาแสดงรายการอาหารในแต่ละรอบ ระบบต้องค้นหาว่ามีรายการใดอยู่ในรอบนั้นบ้าง ดัชนีนี้ช่วยให้ค้นหาตามรหัสรอบได้โดยไม่ต้องไล่ตรวจข้อมูลทุกแถว เหมาะกับกรณีที่มีรายการอาหารสะสมจำนวนมาก

```

66 CREATE INDEX idx_order_items_round_id
67 ON order_items(round_id);
68

```

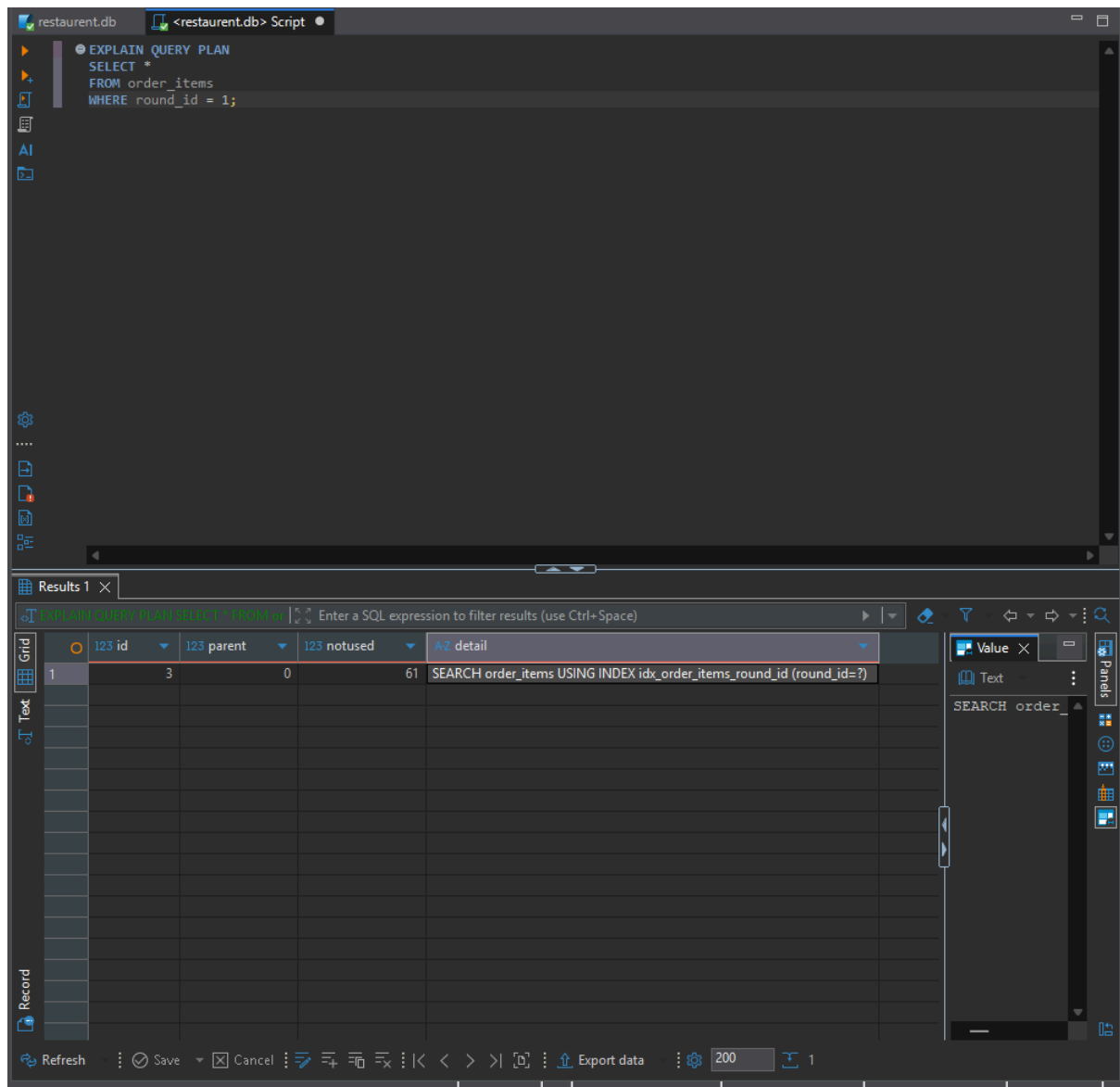
ในไฟล์ schema

ตรวจสอบการใช้งานดัชนีด้วยคำสั่ง:

```

EXPLAIN QUERY PLAN
SELECT *
FROM order_items
WHERE round_id = 1;

```



จากการรันใน DBeaver ข้อความ USING INDEX idx_order_items_round_id แสดงว่าฐานข้อมูลใช้ดัชนีที่สร้างไว้เพื่อค้นหารายการอาหารของรอบรหัส 1 จริง

6.2 ดัชนีที่คอลัมน์ bill_id ของตาราง order_rounds

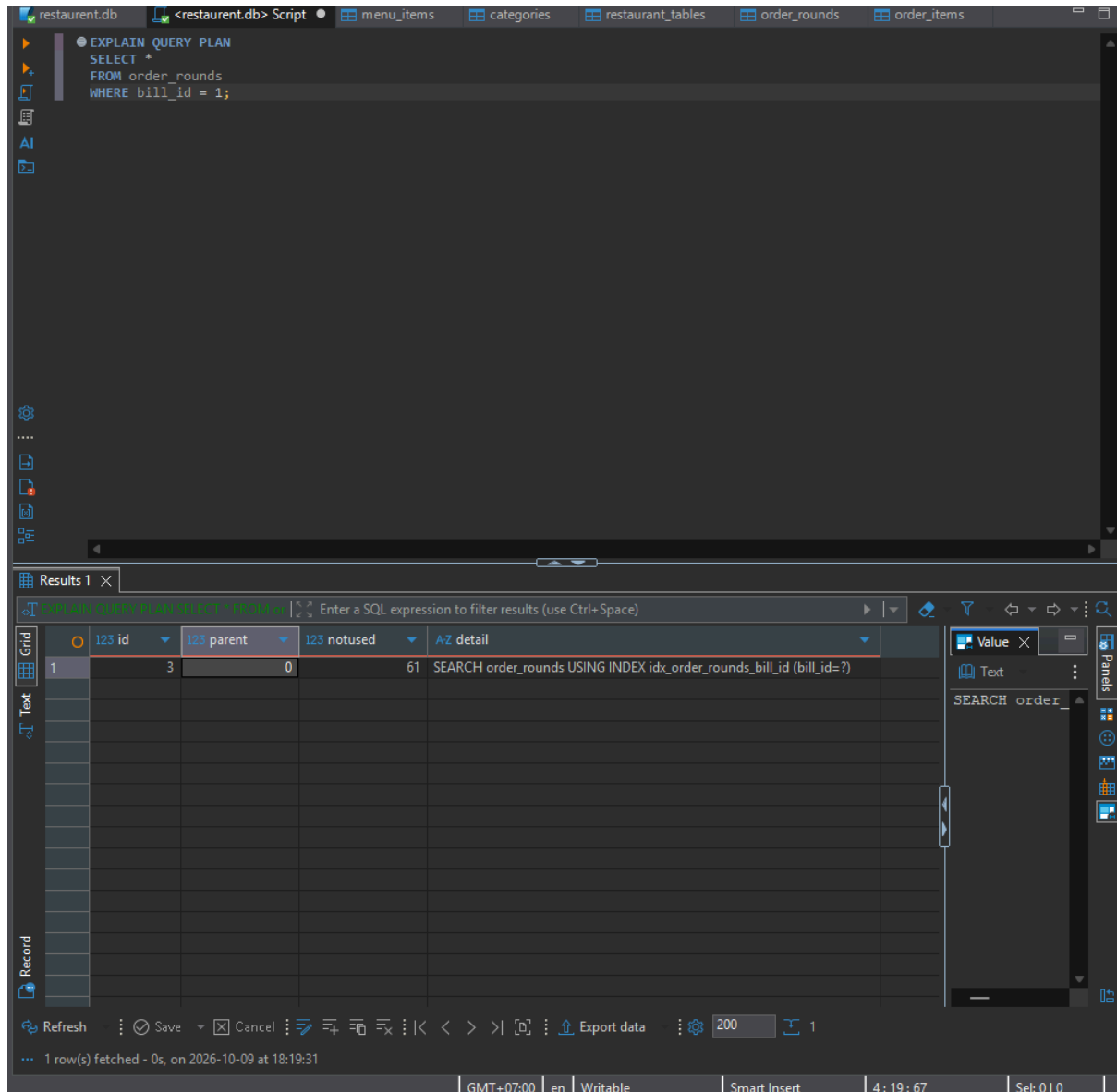
สร้างดัชนีชื่อ idx_order_rounds_bill_id ที่คอลัมน์ bill_id ของตาราง order_rounds เพราะหนึ่งบิลสามารถสั่งอาหารได้หลายรอบ เวลาเปิดดูบิล ระบบจึงต้องค้นหารอบการสั่งทั้งหมดของบิลนั้น ดัชนีนี้ช่วยให้ค้นหาตามรหัสบิลได้โดยไม่ต้องไล่ตรวจรอบการสั่งของทุกบิล

```
68
69 CREATE INDEX idx_order_rounds_bill_id
70 ON order_rounds(bill_id);
```

ในไฟล์ schema

ตรวจสอบการใช้งานดัชนีด้วยคำสั่ง:

```
EXPLAIN QUERY PLAN
SELECT *
FROM order_rounds
WHERE bill_id = 1;
```



จากผลการรันใน DBeaver ข้อความ USING INDEX idx_order_rounds_bill_id แสดงว่าฐานข้อมูลใช้ดัชนีที่สร้างไว้เพื่อค้นหารอบการสั่งทั้งหมดของบิลรหัส 1 จริง

7. ทรานแซกชัน บอกว่าใช้ที่ใดบ้าง และอธิบายว่าถ้าไม่ใช้จะเกิดอะไรขึ้นกับข้อมูล

ระบบใช้ทรานแซกชันในฟังก์ชัน saveOrder() ของไฟล์ restaurantDatabase.js ผ่านคำสั่ง withExclusiveTransactionAsync() เพื่อให้การบันทึกออเดอร์ทำสำเร็จครบทุกขั้นตอน ได้แก่

1. ค้นหาบิลที่เปิดอยู่ของโต๊ะ หากยังไม่มีจะสร้างบิลใหม่ในตาราง bills
2. หาเลขรอบถัดไปและสร้างรอบการสั่งในตาราง order_rounds
3. บันทึกรายการอาหารทั้งหมดของรอบนั้นลงในตาราง order_items

ถ้าทุกขั้นตอนสำเร็จ ระบบจะยืนยันการบันทึกข้อมูล แต่ถ้าเกิดข้อผิดพลาดระหว่างทำงาน จะย้อนกลับ การเปลี่ยนแปลงทั้งหมดในทรานแซกชันครั้งนั้น โดยข้อมูลเดิมก่อนเริ่มยังอยู่เหมือนเดิม

ถ้าไม่ใช้ทรานแซกชัน อาจเกิดการบันทึกข้อมูลไม่ครบ เช่น ลูกค้าสั่งอาหาร 3 รายการ แต่บันทึกได้เพียง 2 รายการแล้วเกิดข้อผิดพลาด ทำให้มีบิลและรอบการสั่งอยู่ แต่รายการอาหารไม่ครบตามที่ลูกค้าสั่ง ส่งผลให้ครัวได้รับรายการไม่ครบและยอดเงินในบิลไม่ตรงกับออเดอร์

- การค้นหา/สร้างบิล และการสร้างรอบการสั่ง

```
120 await db.withExclusiveTransactionAsync(async (transaction) => {
121     let bill = await transaction.getFirstAsync(
122         "SELECT bill_id FROM bills WHERE table_id = ? AND status = 'open' ORDER BY bill_id DESC LIMIT 1",
123         Number(tableId)
124     );
125
126     if (!bill) {
127         const result = await transaction.runAsync(
128             "INSERT INTO bills (table_id, opened_at, status) VALUES (?, ?, 'open')",
129             Number(tableId),
130             new Date().toISOString()
131         );
132         bill = { bill_id: result.lastInsertRowId };
133     }
134
135     const lastRound = await transaction.getFirstAsync(
136         "SELECT COALESCE(MAX(round_number), 0) AS number FROM order_rounds WHERE bill_id = ?",
137         bill.bill_id
138     );
139     const nextRoundNumber = lastRound.number + 1;
140     const newRound = await transaction.runAsync(
141         "INSERT INTO order_rounds (bill_id, round_number, ordered_at) VALUES (?, ?, ?)",
142         bill.bill_id,
143         nextRoundNumber,
144         new Date().toISOString()
145     );
146 }
```

การเริ่มทรานแซกชันและเตรียมบิลกับรอบการสั่ง

- ลูปบันทึกรายการอาหารจนถึงจุดปิดทรานแซกชัน

```
147 for (const item of items) {
148     await transaction.runAsync(
149         "INSERT INTO order_items (round_id, menu_item_id, quantity, unit_price, note, status) VALUES (?, ?, ?, ?, ?, 'waiting')",
150         newRound.lastInsertRowId,
151         Number(item.menu_item_id),
152         Number(item.quantity),
153         Number(item.unit_price),
154         item.note || null
155     );
156 }
157 });
158 }
```

การบันทึกรายการอาหารทั้งหมดภายในทรานแซกชันเดียวกัน

8. ปัญหาที่เจอและวิธีแก้อย่างน้อย 3 เรื่อง แต่ละเรื่องเขียน 3 ส่วน คือ อาการที่เห็น สาเหตุที่แท้จริง และวิธีแก้

8.1 เปลี่ยนราคาเมนูแล้วบิลเก่าเปลี่ยนตาม

- อาการที่เห็น: เมื่อแก้ราคาผัดไทยจาก 50 บาทเป็น 60 บาท บิลที่เคยสั่งตอนราคา 50 บาทกลับแสดงราคาใหม่ ทำให้ยอดเงินย้อนหลังไม่ถูกต้อง

- สาเหตุที่แท้จริง: ตอนแสดงบิล ระบบดึงราคาปัจจุบันจากตาราง menu_items มาคำนวณ โดยไม่ได้เก็บราคาไว้ตอนสั่ง
- วิธีแก้: บันทึกราคาตอนสั่งไว้ในคอลัมน์ unit_price ของตาราง order_items แล้วใช้ราคานี้แสดงและคำนวณยอดบิล ทำให้การเปลี่ยนราคาเมนูไม่กระทบรายการเดิม

8.2 ไม่สามารถกดยกเลิกรายการในตะกร้าได้

- อาการที่เห็น: เมื่อเพิ่มอาหารลงในตะกร้าแล้ว หากเลือกผิดหรือเปลี่ยนใจ ไม่สามารถกดนำรายการนั้นออกก่อนยืนยันการสั่งได้
- สาเหตุที่แท้จริง: ตอนแรกออกแบบให้ตะกร้ารองรับเฉพาะการเพิ่มรายการ แต่ยังไม่ได้ทำปุ่มและฟังก์ชันสำหรับลบรายการออก
- วิธีแก้: เพิ่มปุ่มยกเลิกรายการในตะกร้า เมื่อกดแล้วให้นำรายการที่เลือกออกจากตะกร้าและคำนวณยอดรวมใหม่ ทำให้แก้ไขรายการก่อนยืนยันการสั่งได้

8.3 ไม่สามารถเปลี่ยนชื่อเมนูได้

- อาการที่เห็น: ช่วงแรกเมื่อต้องการเปลี่ยนชื่อเมนู ชื่อที่แสดงในระบบยังเป็นชื่อเดิม ทำให้ไม่สามารถปรับชื่อเมนูตามที่ต้องการได้
- สาเหตุที่แท้จริง: ระบบเพิ่มข้อมูลเมนูเฉพาะตอนที่ยังไม่มีเมนูนั้นในฐานข้อมูล เมื่อแก้ไขชื่อในข้อมูลต้นทาง ระบบจึงไม่ได้อัปเดตชื่อของเมนูที่มีอยู่แล้ว
- วิธีแก้: ปรับคำสั่งบันทึกข้อมูลให้ใช้ ON CONFLICT(menu_item_id) DO UPDATE เพื่ออัปเดตชื่อเมนูเดิม เมื่อแก้ไขชื่อในข้อมูลต้นทางและเปิดระบบให้โหลดข้อมูลใหม่ ชื่อในฐานข้อมูลจึงเปลี่ยนตาม โดยยังใช้รหัสเมนูเดิม

9. การแบ่งงาน ใครทำอะไร พร้อมสัดส่วนโดยประมาณ ต้องสอดคล้องกับประวัติ commit ใน Git

สมาชิก	งานที่รับผิดชอบโดยรวม	สัดส่วนโดยประมาณ
วรมธร เล็กทรัพย์ 6721651785	วางโครงสร้างโปรเจกต์และฐานข้อมูล กำหนดความสัมพันธ์และดัชนี เชื่อมการทำงานระหว่างหน้าจอกับฐานข้อมูล และพัฒนาหน้าเลือกโต๊ะ	34%
ขวัญนนท์ ฐิติเจริญรักษ์ 6721651467	พัฒนาหน้าฝั่งพนักงาน คิวออเดอร์ครัว และหน้าบิลทั้งหมด รวมถึงปรับส่วนเลือกตัวเลือกอาหารและหน้ายืนยันการสั่ง	33%
อิสระ กุมพล 6721651947	พัฒนาและปรับหน้าฝั่งลูกค้า เช่น หน้าเมนู หน้าบิล และหน้าสถานะออเดอร์ รวมถึงปรับรูปแบบหน้าจอให้ใช้งานสะดวก	33%
รวม		100%

10. สิ่งที่จะทำต่อถ้ามีเวลา อย่างน้อย 3 ข้อ

10.1 เพิ่มหน้าจัดการเมนูสำหรับเจ้าของร้าน

ให้เพิ่มเมนู เปลี่ยนชื่อ ปรับราคา และตั้งสถานะอาหารหมดผ่านแอปได้โดยตรง เพื่อไม่ต้องเข้าไปแก้ข้อมูลในโค้ด

10.2 เพิ่มระบบเข้าสู่ระบบและแบ่งสิทธิ์ผู้ใช้

แบ่งสิทธิ์เป็นเจ้าของร้าน พนักงาน และครัว เพื่อให้แต่ละคนเข้าถึงเฉพาะส่วนที่เกี่ยวข้องกับหน้าที่ของตัวเอง

10.3 ปรับปรุงหน้าตาแอป UI ให้สวยและใช้งานง่ายขึ้น

ปรับสี ตัวอักษร และระยะห่างให้สม่ำเสมอ เพิ่มขนาดปุ่มให้กดสะดวก รวมถึงปรับหน้าจอให้แสดงผลบน Tablet